



GESTÃO DE CLUSTERS,
SEGURANÇA QUÂNTICA E
COLABORAÇÃO MULTIAGENTE

QUANDO A AUTOMAÇÃO APRENDE: INTEGRANDO IA E MACHINE LEARNING NO RPA



LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Instalação das bibliotecas (1)	9
Comando de prompt 2 - Instalação das bibliotecas caso Windows.....	9
Comando de prompt 3 - Instalação das bibliotecas (2)	23
Comando de prompt 4 - Instalação das bibliotecas (3)	23

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Código do leitor / Extrator de Danfe.....	14
Código-fonte 2 - Exemplo de código-fonte HTML	26
Código-fonte 3 - Pipeline Completo.....	32

EXEMPLO

SUMÁRIO

1 QUANDO A AUTOMAÇÃO APRENDE: INTEGRANDO IA E MACHINE LEARNING NO RPA.....	5
2 PROCESSAMENTO INTELIGENTE DE DOCUMENTOS (IDP)	8
3 MOTOR INFERENCIAL NA HIPERAUTOMAÇÃO E SUA EXPLICABILIDADE	19
3.1 Funcionamento de Experta para implementação de motores inferenciais	20
3.2 Contextualização do exemplo integrado.....	21
3.3 Instalação e preparação do ambiente	23
3.4 Código 1: motor inferencial com Experta.....	24
3.5 Pipeline de automação com OCR, ML e inferência.....	27
CONCLUSÃO.....	34
REFERÊNCIAS.....	36

1 QUANDO A AUTOMAÇÃO APRENDE: INTEGRANDO IA E MACHINE LEARNING NO RPA

A crescente digitalização de processos organizacionais tem transformado profundamente a forma como as informações são produzidas, armazenadas e utilizadas. Em praticamente todos os setores, incluindo indústria, finanças, educação e serviços, uma grande quantidade de dados é gerada diariamente na forma de documentos, registros operacionais e relatórios. Esses dados frequentemente contêm informações relevantes que precisam ser analisadas, classificadas e utilizadas para orientar decisões e ações. No entanto, a análise manual desses registros pode ser lenta, sujeita a erros e limitada pela capacidade humana de processar grandes volumes de informação em tempo reduzido. Esse cenário tem impulsionado o desenvolvimento de sistemas automatizados capazes de processar dados de forma eficiente e consistente.

Nesse contexto, a Automação Robótica de Processos, conhecida como RPA (Robotic Process Automation), surge como uma abordagem que permite automatizar tarefas repetitivas e baseadas em regras, tradicionalmente realizadas por operadores humanos. Sistemas de RPA são capazes de interagir com arquivos, sistemas operacionais e aplicações digitais, executando tarefas como leitura de documentos, extração de informações e execução de ações predefinidas. Esses sistemas operam de forma determinística, seguindo regras explícitas programadas previamente. Embora esse modelo seja eficaz para tarefas estruturadas e previsíveis, ele apresenta limitações quando aplicado a cenários que envolvem dados não estruturados ou ambíguos, como texto livre ou documentos cujo conteúdo pode variar significativamente.

A integração da Inteligência Artificial com sistemas de automação representa um avanço significativo nesse cenário. Técnicas de aprendizado de máquina permitem que sistemas automatizados analisem dados e identifiquem padrões que não podem ser facilmente capturados por regras fixas. Por meio do treinamento com exemplos, esses modelos podem aprender a classificar documentos, reconhecer padrões textuais e estimar a probabilidade de diferentes categorias ou condições. Essa capacidade amplia o escopo da automação, permitindo que sistemas lidem com

informações não estruturadas e tomem decisões baseadas em evidências extraídas dos dados.

Entretanto, modelos de aprendizado de máquina produzem estimativas probabilísticas, não decisões operacionais definitivas. O resultado de um modelo é, em geral, uma previsão acompanhada de um nível de confiança, indicando o grau de certeza da estimativa. Esse tipo de saída é útil, mas não suficiente para determinar diretamente qual ação deve ser executada em um sistema automatizado. Em ambientes organizacionais, é necessário aplicar critérios adicionais para decidir como agir com base nessas estimativas. Por exemplo, pode ser necessário definir limiares de confiança ou considerar outras informações disponíveis antes de executar uma ação automatizada. Esse processo requer a aplicação de lógica explícita e estruturada.

É nesse ponto que os motores inferenciais desempenham um papel fundamental. Um motor inferencial é um componente capaz de aplicar regras lógicas sobre um conjunto de fatos para produzir conclusões ou decisões. Esses sistemas são baseados em princípios da lógica formal e permitem representar conhecimento na forma de regras do tipo “se-então”. Ao receber informações sobre um determinado caso, o motor inferencial avalia quais regras são aplicáveis e produz inferências com base nessas regras. Esse mecanismo permite transformar informações em decisões operacionais de forma estruturada, transparente e explicável. A utilização de motores inferenciais possibilita separar o processo de estimativa, realizado por modelos estatísticos, do processo de decisão, realizado por regras explícitas.

A combinação de aprendizado de máquina e motores inferenciais resulta em uma arquitetura híbrida que integra capacidades estatísticas e simbólicas. O modelo de aprendizado de máquina é responsável por analisar os dados e produzir estimativas com base em padrões aprendidos. O motor inferencial, por sua vez, utiliza essas estimativas como fatos e aplica regras para determinar qual ação deve ser executada. Essa separação permite que o sistema mantenha flexibilidade na análise de dados e, ao mesmo tempo, preserve controle e explicabilidade no processo de decisão. Além disso, as regras podem ser modificadas independentemente do modelo estatístico, permitindo adaptar o comportamento do sistema conforme necessário.

A implementação prática desses sistemas envolve múltiplos componentes que operam em conjunto. Inicialmente, é necessário adquirir os dados, o que pode envolver a leitura de arquivos ou a captura de informações de sistemas existentes. Em seguida, os dados devem ser processados e transformados em uma forma adequada para análise, o que pode incluir etapas como limpeza, normalização e extração de características. O modelo de aprendizado de máquina utiliza essas características para produzir uma estimativa. O motor inferencial recebe essa estimativa, juntamente com outras informações relevantes, e aplica regras para determinar a ação apropriada. Por fim, o sistema executa a ação definida, como mover um arquivo, gerar um relatório ou atualizar um sistema.

Um aspecto importante dessa abordagem é a explicabilidade das decisões. Diferentemente de sistemas puramente estatísticos, nos quais o processo de decisão pode ser difícil de interpretar, sistemas que utilizam motores inferenciais permitem identificar claramente quais regras foram aplicadas e por que uma determinada decisão foi tomada. Essa característica é essencial em ambientes nos quais é necessário compreender e justificar o comportamento do sistema. A explicabilidade contribui para aumentar a confiança no sistema e facilita sua validação e manutenção.

A linguagem Python desempenha um papel central na implementação desses sistemas, devido à sua simplicidade, flexibilidade e amplo ecossistema de bibliotecas. Bibliotecas como scikit-learn permitem implementar modelos de aprendizado de máquina de forma eficiente, enquanto bibliotecas como experta possibilitam a criação de motores inferenciais baseados em regras. Além disso, as bibliotecas padrão do Python fornecem ferramentas para manipulação de arquivos, processamento de texto e automação de tarefas no sistema operacional. Essa combinação torna o Python uma escolha adequada para o desenvolvimento de sistemas de automação inteligente.

Neste capítulo, será apresentada a construção de um sistema completo que integra aprendizado de máquina, motores inferenciais e automação utilizando Python. O objetivo é demonstrar, de forma prática, como essas tecnologias podem ser utilizadas para processar dados, tomar decisões automatizadas e executar ações operacionais. Ao longo do capítulo, serão explorados os conceitos fundamentais e sua aplicação em um sistema funcional, permitindo que os estudantes compreendam tanto os aspectos teóricos quanto os práticos dessa abordagem.

2 PROCESSAMENTO INTELIGENTE DE DOCUMENTOS (IDP)

Grande parte da informação corporativa não está organizada em bancos de dados, mas em documentos: PDFs, imagens, contratos, relatórios, recibos, e-mails e notas fiscais. Esses documentos contêm dados críticos para a operação de qualquer empresa, mas são de difícil acesso automatizado. Um robô RPA tradicional, projetado para manipular telas ou APIs, não consegue compreender o conteúdo de uma nota fiscal digitalizada nem extrair valores ou datas de um PDF. Surge então o desafio: **como automatizar fluxos que dependem da leitura de documentos não estruturados**, preservando a confiabilidade e evitando trabalho manual humano?

Esse problema é conhecido como **Processamento Inteligente de Documentos (Intelligent Document Processing – IDP)**. Ele representa uma evolução natural da RPA clássica, integrando técnicas de visão computacional e processamento de linguagem natural (NLP). O objetivo é dar aos robôs a capacidade de “ler e entender” documentos como humanos fariam, mas com velocidade e precisão de máquina. No contexto empresarial, o IDP é a peça-chave que permite transformar dados não estruturados em informações úteis — e, principalmente, integrá-las automaticamente aos sistemas corporativos.

Um dos exemplos mais emblemáticos desse tipo de automação é a **extração de dados de Notas Fiscais Eletrônicas (NF-e)**. O documento fiscal é um caso de uso real e recorrente: chega em PDF, vem de diversos fornecedores, e cada um usa um layout diferente. Apesar da padronização do XML, o que o departamento financeiro geralmente recebe é o **PDF escaneado da NF**, que precisa ser processado manualmente por alguém. Essa tarefa, embora repetitiva, exige precisão e está sujeita a erros humanos. Um robô que consiga **ler o PDF, identificar automaticamente o cabeçalho e os itens de produto, e exportar tudo para um CSV** oferece ganho direto de produtividade e confiabilidade.

Além de eliminar o trabalho manual, a automação desse tipo de leitura cria uma base de dados consistente que pode alimentar sistemas de ERP, relatórios gerenciais e rotinas de compliance. Mas o desafio técnico é substancial: o OCR (Optical Character Recognition) precisa lidar com diferentes qualidades de imagem; os textos extraídos vêm com ruídos; os rótulos variam (“Razão Social”, “Nome / Razão Social”,

“Emitente”) e o formato dos números nem sempre segue um padrão fixo. Resolver esse problema requer um pipeline que combine **reconhecimento visual, expressões regulares, busca aproximada e compreensão semântica** — um exemplo clássico de **IA aplicada à automação de processos de negócio**.

Por isso, neste primeiro projeto prático, construímos um **extrator inteligente de notas fiscais** que percorre automaticamente uma pasta de PDFs, faz OCR de cada documento, extrai o **cabeçalho** (CNPJ, razão social, série, valor, natureza da operação etc.), identifica e organiza **todos os itens de produto** em ordem, e salva os resultados em arquivos CSV distintos. Essa é uma aplicação concreta e poderosa de IA para RPA, pois conecta os mundos da visão computacional (OCR), NLP (interpretação textual) e automação (integração de dados).

A seguir, apresentamos o código completo desse projeto, implementado em Python, com bibliotecas amplamente utilizadas no mercado — pytesseract, pdf2image, spaCy e Pandas. Ele demonstra como, com uma arquitetura bem estruturada e técnicas simples de IA, é possível transformar uma tarefa tediosa e manual em um processo autônomo e escalável. Não se esqueça de executar a instalação do ambiente previamente com:

```
pip install pandas pytesseract pdf2image spacy pillow  
python -m spacy download pt_core_news_sm
```

Comando de prompt 1 - Instalação das bibliotecas (1)
Fonte: Elaborado pelo autor (2026)

Caso sistema Linux, use:

```
sudo apt install tesseract-ocr tesseract-ocr-por
```

Comando de prompt 2 - Instalação das bibliotecas caso Windows
Fonte: Elaborado pelo autor (2026)

Caso utilize Windows, o download do Tesseract está disponível no repositório da universidade de Mannheim <<https://github.com/UB-Mannheim/tesseract>>.

```

import os
import re
import pandas as pd
import pytesseract
from pdf2image import convert_from_path
import spacy
import unicodedata
from difflib import SequenceMatcher

=====
CONFIGURAÇÕES INICIAIS
=====

Observação: instale o modelo antes com:
python -m spacy download pt_core_news_sm
E garanta o Tesseract com o idioma 'por' instalado.
nlp = spacy.load("pt_core_news_sm")
def normalizar_texto(txt): if not txt: return "" txt =
txt.lower() txt = unicodedata.normalize("NFKD", txt) txt =
"".join([c for c in txt if not unicodedata.combining(c)]) txt =
re.sub(r'[^a-z0-9\s/]', ' ', txt) txt = re.sub(r'\s+', ' ',
txt) return txt.strip()
def similar(a, b): return SequenceMatcher(None,
normalizar_texto(a), normalizar_texto(b)).ratio()
=====
FUNÇÃO OCR
=====

def extrair_texto_pdf(caminho_pdf): """ Converte um
arquivo PDF em imagens e aplica OCR para extrair o texto. """
paginas = convert_from_path(caminho_pdf, dpi=300) texto = ""
for pagina in paginas: # Se 'por' não estiver instalado,
troque para 'eng' temporariamente texto +=
pytesseract.image_to_string(pagina, lang='por') + "\n"
return texto
=====
FUNÇÃO DE BUSCA FLEXÍVEL POR LABELS
=====

def buscar_por_label(texto, lista_labels, janela=60):
texto_norm = normalizar_texto(texto) for label in
lista_labels: label_norm = normalizar_texto(label) # Busca
aproximada tolerante a OCR for match in
re.finditer(label_norm[:6], texto_norm): inicio =
match.start() trecho = texto_norm[inicio : inicio +
len(label_norm) + janela] if similar(trecho, label_norm) >
0.7: padrao_valor = re.search(label_norm + r'[:/-]*([a-z0-
9\s.-&]+)', trecho) if padrao_valor: return
padrao_valor.group(1).strip() return None
=====
EXTRAÇÃO DO CABEÇALHO
=====

def extrair_entidades(texto): """ Extrai campos
principais de uma NF com tolerância fonética e estrutural.
""" texto_limpo = normalizar_texto(texto)

```

```

# Padrões fixos
padrao_cnpj = re.search(r'\d{2}\.\d{3}\.\d{3}/\d{4}-\d{2}',
texto)
padrao_chave = re.search(r'(\d[\d\s]{40,})', texto)
padrao_valor = re.search(r'r?\$ ?\d{1,3}(?:\.\d{3})*,\d{2}',
texto, re.IGNORECASE)
padrao_data = re.search(r'\b(?:[0-3]?\d[/.-][01]?\d[/.-]
)(?:\d{4}|\d{2}))\b', texto)
padrao_numero = re.search(r'(?n[°o]|numero|nf) [\s:]*([0-
9]{3,})', texto_limpo)
padrao_serie = re.search(r'serie[:\s]*([0-9a-z]+)',
texto_limpo)

# Sinonímias e variações estruturais
labels = {
    "razao_social": ["razao social", "nome / razao social",
"emitente", "empresa", "nome razao social", "nome do
emitente"],
    "natureza_operacao": ["natureza da operacao", "finalidade
da operacao", "tipo de operacao", "natureza op", "natureza da
operação"],
    "inscricao_estadual": ["inscricao estadual", "i e", "ie",
"inscr estadual"]
}

razao = buscar_por_label(texto, labels["razao_social"])
natureza = buscar_por_label(texto,
labels["natureza_operacao"])
inscricao = buscar_por_label(texto,
labels["inscricao_estadual"])

# Fallback semântico com spaCy
if not razao:
    doc = nlp(texto)
    for ent in doc.ents:
        if ent.label_ in ["ORG"]:
            razao = ent.text.strip()
            break

return {
    "CNPJ Emitente": padrao_cnpj.group(0) if padrao_cnpj else
None,
    "Razão Social": razao,
    "Número NF": padrao_numero.group(1) if padrao_numero else
None,
    "Série": padrao_serie.group(1) if padrao_serie else None,
    "Data de Emissão": padrao_data.group(0) if padrao_data
else None,
    "Valor Total": padrao_valor.group(0) if padrao_valor else
None,
    "Natureza da Operação": natureza,

```

```

    "Chave de Acesso": re.sub(r'\s+', ' ',
padrao_chave.group(1)) if padrao_chave else None,
    "Inscrição Estadual": inscricao
}

=====
EXTRAÇÃO DOS ITENS
=====

def extrair_itens(texto): """ Extrai itens do
detalhamento da NF-e. Retorna uma lista de dicionários:
Código, Descrição, Quantidade, Unidade, Valor Unitário.
Funciona mesmo sem títulos explícitos por meio de
heurísticas. """ texto_limpo = re.sub(r'[^\\w\\s,.-/]', ' ',
texto) linhas = [l.strip() for l in texto_limpo.split("\n")
if l.strip()]
    # 1) Procurar início/fim de forma flexível
    inicio_idx, fim_idx = None, None
    for i, linha in enumerate(linhas):
        ln = normalizar_texto(linha)
        if any(p in ln for p in [
            "itens da nota", "itens nota", "detalhamento dos
produtos",
            "dados dos produtos", "produto servico", "cod do
produto",
            "codigo descricao qtde", "descricao dos produtos"
        ]):
            inicio_idx = i + 1
        if any(p in ln for p in [
            "total da nota", "resumo dos valores", "valores
totais", "total geral"
        ]):
            fim_idx = i
            break

    # 2) Fallback por padrão de linhas com números/valores
    if inicio_idx is None:
        for i, linha in enumerate(linhas):
            if re.search(r'^\d{1,5}\s+[A-
Z].+\d{1,3}(?:[.,]\d{3})*,\d{2}', linha):
                inicio_idx = max(i - 1, 0)
                break

    if fim_idx is None:
        fim_idx = len(linhas)

    if inicio_idx is None:
        # Detecção por densidade numérica
        blocos = []
        bloco = []
        for l in linhas:
            if re.search(r'\d', l):

```

```

        bloco.append(l)
    elif bloco:
        blocos.append(bloco)
        bloco = []
    if bloco:
        blocos.append(bloco)

    if blocos:
        blocos_validos = [b for b in blocos if len([l for l
in b if re.search(r'\d{1,3},\d{2}', l)]) >= 3]
        if blocos_validos:
            bloco_itens = max(blocos_validos, key=len)
        else:
            return []
    else:
        return []
else:
    bloco_itens = linhas[inicio_idx:fim_idx]

# 3) Agrupar linhas quebradas pelo OCR
itens = []
item_atual = ""
for linha in bloco_itens:
    if re.match(r'^\d{1,6}\s', linha): # nova linha de item
        indica início por código
        if item_atual:
            itens.append(item_atual.strip())
            item_atual = linha
        else:
            item_atual += " " + linha
if item_atual:
    itens.append(item_atual.strip())

# 4) Extrair campos
registros = []
for item in itens:
    codigo = re.search(r'^\d{1,6}', item)
    qtd =
re.search(r'(\d+(?:[.,]\d+)?)\s*(UN|KG|LT|CX|PC|PÇ|UNID) ',
item, re.IGNORECASE)
    valores = re.findall(r'(\d{1,3}(?:\.\d{3})*,\d{2})',
item)
    descricao = re.sub(r'^\d{1,6}\s+', '', item)
    descricao = re.sub(r'\s{2,}', ' ', descricao)

    registros.append({
        "Código": codigo.group(0) if codigo else None,
        "Descrição": descricao.strip() if descricao else
None,
        "Quantidade": qtd.group(1) if qtd else None,
        "Unidade": (qtd.group(2).upper() if qtd else None),
    })

```

```
        "Valor Unitário": (valores[-1] if valores else None)
    })

return registros

=====
SALVAR CSV DE ITENS
=====

def salvar_itens_csv(itens,
arquivo_saida="itens_nota.csv"): if not itens: print("Nenhum
item encontrado para salvar.") return df =
pd.DataFrame(itens) df.to_csv(arquivo_saida, index=False,
sep=";") print(f"Itens salvos em {arquivo_saida} ({len(df)}
linhas).")
=====
PIPELINE PRINCIPAL
=====

def processar_pasta(diretorio): resultados = [] for
arquivo in os.listdir(diretorio): if
arquivo.lower().endswith(".pdf"): caminho =
os.path.join(diretorio, arquivo) print(f"\nProcessando
{arquivo}...") try: texto = extrair_texto_pdf(caminho) dados
= extrair_entidades(texto) dados["Arquivo"] = arquivo
resultados.append(dados)
        itens = extrair_itens(texto)
        if itens:
            salvar_itens_csv(itens,
arquivo_saida=f"itens_{os.path.splitext(arquivo)[0]}.csv")

    except Exception as e:
        print(f"Erro ao processar {arquivo}: {e}")
return pd.DataFrame(resultados)

=====
EXECUÇÃO PRINCIPAL
=====

if name == "main": pasta = "notas_fiscais" saida =
"dados_extraidos.csv"
    if not os.path.exists(pasta):
        print(f"Pasta '{pasta}' não encontrada. Crie uma pasta
chamada '{pasta}' e coloque PDFs dentro dela.")
    else:
        df = processar_pasta(pasta)
        if not df.empty:
            df.to_csv(saida, index=False, sep=";")
            print(f"\nCabeçalhos extraídos e salvos em {saida}")
        else:
            print("\nNenhum dado foi extraído. Verifique os PDFs
e o OCR.")
```

Código-fonte 1 - Código do leitor / Extrator de Danfe

Fonte: Elaborado pelo autor (2025)

O script completo de extração de dados de notas fiscais eletrônicas é um exemplo didático de integração entre técnicas clássicas de automação e componentes de Inteligência Artificial. Seu funcionamento parte de um problema prático recorrente em empresas: receber centenas de documentos fiscais em formato PDF, emitidos por diferentes fornecedores, com variações visuais, acentuação inconsistente e até falhas de digitalização. O objetivo do programa é automatizar a leitura, a interpretação e a organização dessas informações, reduzindo o trabalho manual e aumentando a confiabilidade do processo.

O processamento inicia-se com a leitura dos arquivos PDF. Como grande parte das notas chega em formato de imagem e não em texto digital, é necessário convertê-las em imagens rasterizadas. Essa tarefa é executada pela biblioteca **pdf2image**, que utiliza internamente o motor *Poppler* para renderizar cada página em alta resolução. A escolha de uma resolução de 300 dpi não é arbitrária: trata-se de um equilíbrio entre legibilidade e custo computacional. A seguir, cada imagem é enviada ao **pytesseract**, um wrapper em Python para o motor OCR Tesseract, que aplica redes neurais LSTM (Long Short-Term Memory) para reconhecer sequências de caracteres mesmo em imagens imperfeitas. Quando o idioma português está disponível (parâmetro `lang='por'`), o modelo usa um dicionário léxico e estatístico que melhora a acurácia sobre palavras com acentos e grafias típicas do português, como “operação” ou “razão”.

O texto obtido pelo OCR é ruidoso e inconsistente. Por isso, o script aplica uma etapa de **normalização textual**, uma prática comum em pipelines de NLP (Natural Language Processing). Essa fase utiliza funções da biblioteca `unicodedata` para decompor caracteres acentuados em sua forma base e remover os diacríticos. O texto é então convertido para minúsculas e filtrado por expressões regulares que eliminam símbolos desnecessários e reduzem espaços consecutivos. Esse processo tem como finalidade produzir uma representação canônica e uniforme, tornando o texto mais previsível para os algoritmos de busca e extração que virão em seguida. A normalização é uma forma simples, porém eficaz, de reduzir o impacto dos erros de OCR e das variações ortográficas.

A extração das informações do cabeçalho da nota fiscal é feita por dois mecanismos complementares: **reconhecimento estrutural por padrões** e **busca aproximada por rótulos**. O reconhecimento estrutural usa expressões regulares

cuidadosamente elaboradas para detectar formatos específicos de campos previsíveis, como o CNPJ, as datas e os valores monetários. A expressão `\d{2}\.\d{3}\.\d{3}\d{4}-\d{2}`, por exemplo, captura o formato exato de um CNPJ, independente do texto ao redor. Da mesma forma, o padrão `R?\$?\d{1,3}(?:\.\d{3})*\d{2}` encontra valores em real no formato brasileiro, tolerando espaços e pontos de milhar. Essa etapa é determinística e altamente confiável quando o OCR é legível.

Para campos textuais e mais variáveis, como “Razão Social” ou “Natureza da Operação”, o código emprega uma estratégia mais flexível. Ele mantém um dicionário com possíveis variações de cada rótulo — por exemplo, “Razão Social”, “Nome / Razão Social”, “Emitente”, “Empresa” — e tenta localizar essas expressões no texto de forma aproximada, utilizando o algoritmo `SequenceMatcher` da biblioteca **difflib**. Esse algoritmo calcula a similaridade entre duas sequências de caracteres com base em subsequências coincidentes, retornando um valor entre 0 e 1. Se a semelhança entre um rótulo esperado e o trecho do texto for superior a um limiar (geralmente 0,7), considera-se que o rótulo foi encontrado. Em seguida, uma pequena janela de caracteres após o rótulo é analisada para capturar o valor associado. Essa técnica de *fuzzy matching* permite tolerar distorções comuns de OCR, como “RAZAO SOCIAL” (com L e I trocados) ou “NOME-RAZAO SOCiAL”.

Quando mesmo essa busca falha, entra em cena uma camada de **compreensão semântica leve** com a biblioteca **spaCy**. O texto completo é analisado linguisticamente, e o modelo pré-treinado `pt_core_news_sm` tenta reconhecer entidades nomeadas — pessoas, organizações, locais e datas. Se o modelo identificar uma entidade do tipo `ORG`, ela é usada como possível valor para a razão social. Essa é a primeira inserção direta de IA no pipeline: em vez de depender apenas de padrões ou rótulos explícitos, o sistema passa a inferir o significado de palavras com base em contexto. Embora simples, esse fallback é poderoso, pois cobre casos em que o documento traz apenas o nome da empresa em destaque, sem qualquer indicação textual de “Razão Social”.

A segunda parte do pipeline lida com o **detalhamento dos itens** da nota. O problema aqui é mais complexo, pois o layout varia amplamente entre fornecedores e muitos PDFs não trazem cabeçalhos de tabela consistentes. A função de extração tenta identificar o bloco de itens de duas maneiras. Primeiro, busca por palavras-chave

normalizadas que possam indicar o início ou o fim da tabela — termos como “itens da nota”, “detalhamento dos produtos” ou “resumo dos valores”. Se esses rótulos não existirem, o algoritmo recorre a um método heurístico baseado em **densidade numérica**: percorre o texto linha a linha, identifica blocos com alta concentração de números e valores monetários e escolhe aquele com mais linhas como provável seção de itens. Essa abordagem é uma forma de *feature extraction* manual, pois usa características estruturais — presença de dígitos, separadores, vírgulas — como indícios de que um segmento de texto representa uma tabela.

Após delimitar o bloco, o código realiza o **agrupamento de linhas**. O OCR frequentemente quebra uma linha de produto em duas ou mais linhas, especialmente quando a tabela é densa ou o espaçamento é irregular. O script considera que uma nova linha de item começa quando a linha se inicia com um número (geralmente o código do produto). As linhas seguintes, que não possuem esse padrão, são concatenadas ao item anterior. Esse procedimento reconstrói o texto original de cada linha de tabela e evita a perda de informações fragmentadas.

A extração dos campos de cada item (código, descrição, quantidade, unidade e valor) é novamente baseada em expressões regulares, mas agora aplicadas dentro de um contexto delimitado. O código de produto é identificado no início da linha, a quantidade e a unidade são capturadas por padrões que reconhecem as abreviações mais comuns (UN, KG, CX, PC, PÇ), e os valores são extraídos considerando formatos monetários válidos. Como cada item pode conter tanto valor unitário quanto valor total, o script opta por capturar o último número monetário encontrado na linha — uma simplificação prática para documentos homogêneos.

Do ponto de vista conceitual, essa segunda etapa combina técnicas de **IA simbólica** (regras e padrões) e **aprendizado estatístico** (reconhecimento de entidades). Embora não haja um modelo de aprendizado treinado especificamente, há uma lógica de inferência embutida: o sistema interpreta a estrutura do texto e aplica heurísticas de decisão, tal como um humano faria ao “enxergar” uma tabela. Esse tipo de raciocínio híbrido é a essência da hyperautomation ou seja, o uso combinado de automação robótica, NLP e algoritmos inteligentes para entender e agir sobre dados.

Por fim, os dados extraídos são organizados em estruturas DataFrame da biblioteca **pandas**, que facilita a manipulação tabular e a exportação para CSV. O

arquivo dados_extraídos.csv contém um resumo com os cabeçalhos de todas as notas processadas, enquanto cada conjunto de itens é salvo em arquivos individuais com o prefixo itens_. Essa separação preserva o vínculo entre o cabeçalho e seu detalhamento e permite que o robô ou outro sistema RPA realize integrações posteriores com ERPs, bancos de dados ou planilhas corporativas.

De modo geral, o script demonstra que, com bibliotecas abertas e técnicas relativamente simples de IA, é possível construir soluções robustas para problemas reais de automação documental. O segredo está na combinação das abordagens: o OCR fornece a base perceptiva, a normalização e as regex dão estrutura, o *fuzzy matching* traz tolerância a ruído e o NLP oferece compreensão contextual. Juntas, essas camadas fazem o robô evoluir de um executor de tarefas fixas para um agente capaz de interpretar e decidir — o ponto de partida para sistemas de automação cognitiva verdadeiramente inteligentes.

3 MOTOR INFERENCIAL NA HIPERAUTOMAÇÃO E SUA EXPLICABILIDADE

Sistemas de hiperautomação que incorporam Inteligência Artificial frequentemente combinam dois paradigmas complementares de tomada de decisão. O paradigma estatístico utiliza modelos de Machine Learning para estimar probabilidades com base em dados observados, enquanto o paradigma simbólico utiliza motores inferenciais baseados em regras declarativas para derivar conclusões a partir de fatos conhecidos. O motor inferencial representa o componente responsável por transformar informações brutas e estimativas probabilísticas em decisões operacionais explícitas e justificáveis, atuando como um mecanismo formal de raciocínio dentro do pipeline de automação.

O funcionamento de um motor inferencial é baseado na interação entre uma base de fatos e uma base de regras. A base de fatos representa o estado atual do sistema, contendo informações obtidas a partir de fontes externas ou produzidas por modelos estatísticos. Esses fatos podem incluir, por exemplo, a categoria estimada de um documento, o nível de confiança dessa estimativa, valores extraídos de conteúdo textual e outros atributos relevantes. A base de regras contém expressões lógicas que definem como novos fatos ou decisões devem ser derivados a partir dos fatos existentes. Cada regra estabelece condições e ações associadas, permitindo que o sistema realize inferências estruturadas.

O mecanismo de inferência opera por meio de um processo denominado encadeamento progressivo, no qual o sistema avalia continuamente as regras disponíveis e verifica quais condições são satisfeitas pelos fatos presentes na memória operacional. Quando uma regra é ativada, suas ações produzem novos fatos ou decisões que são adicionados à memória. Esses novos fatos podem ativar regras adicionais, permitindo que o sistema construa uma sequência de inferências sucessivas até atingir um estado estável. Esse comportamento permite que decisões complexas sejam construídas gradualmente a partir de informações iniciais simples.

Um aspecto fundamental dos motores inferenciais é a sua capacidade de fornecer explicabilidade das decisões automatizadas. Diferentemente de muitos modelos estatísticos que produzem apenas uma previsão final, o motor inferencial

mantém um registro explícito das regras que foram ativadas e dos fatos que foram utilizados para produzir uma decisão. Isso permite reconstruir o caminho lógico que levou a uma determinada ação. Essa propriedade é essencial em sistemas corporativos e regulados, onde é necessário justificar decisões automatizadas e permitir auditorias. A explicabilidade também facilita o diagnóstico de comportamentos inesperados e permite que especialistas ajustem o sistema modificando regras específicas, sem alterar o restante do pipeline.

A integração entre modelos estatísticos e motores inferenciais permite combinar capacidade preditiva com transparência decisória. O modelo de Machine Learning fornece uma estimativa baseada em padrões aprendidos a partir de dados históricos, enquanto o motor inferencial interpreta essa estimativa dentro de um conjunto de regras explícitas. Essa arquitetura híbrida permite que o sistema tome decisões consistentes e auditáveis, mantendo ao mesmo tempo a capacidade de adaptação proporcionada pelo aprendizado de máquina.

3.1 Funcionamento de Experta para implementação de motores inferenciais

A biblioteca Experta é um framework em Python projetado para implementar sistemas especialistas baseados em regras e encadeamento progressivo. Seu funcionamento é fundamentado no conceito de memória operacional, regras declarativas e um mecanismo de inferência que executa automaticamente o processo de raciocínio. A biblioteca permite que desenvolvedores definam fatos e regras de maneira estruturada, delegando ao motor inferencial a responsabilidade de avaliar condições e executar inferências.

O primeiro componente fundamental da biblioteca é a definição de fatos, que representam informações sobre o estado do sistema. Fatos são definidos como objetos que encapsulam atributos relevantes para o processo de decisão. Esses objetos são inseridos na memória operacional do motor inferencial e servem como base para a avaliação das regras. A memória operacional representa o estado atual do sistema e é atualizada dinamicamente conforme novas inferências são produzidas.

O segundo componente consiste na definição das regras inferenciais. Cada regra é composta por um conjunto de condições e um conjunto de ações. As condições especificam os padrões que devem ser satisfeitos pelos fatos presentes na

memória operacional, enquanto as ações definem quais novos fatos ou decisões devem ser produzidos quando a regra é ativada. A biblioteca permite definir regras de forma declarativa, utilizando estruturas que expressam diretamente a lógica de decisão. Esse modelo facilita a manutenção e torna a lógica do sistema mais transparente.

O motor inferencial da biblioteca é responsável por executar o processo de encadeamento progressivo. Após a inserção dos fatos iniciais, o motor analisa todas as regras definidas e identifica aquelas cujas condições são satisfeitas. Quando uma regra é ativada, suas ações são executadas e novos fatos são adicionados à memória operacional. Esse processo continua automaticamente até que nenhuma nova regra possa ser ativada. O motor mantém o estado completo do processo, permitindo que o sistema derive conclusões complexas a partir de um conjunto inicial de fatos.

A biblioteca também facilita a implementação de sistemas explicáveis, pois mantém explicitamente os fatos e decisões produzidos durante o processo de inferência. Isso permite que o sistema registre quais condições foram satisfeitas e quais regras foram ativadas. Esse comportamento é essencial para sistemas de automação que exigem rastreabilidade e auditoria. A capacidade de observar diretamente os fatos e inferências produzidos permite compreender o comportamento do sistema e ajustar suas regras conforme necessário.

A integração da biblioteca Experta com pipelines de automação permite construir sistemas nos quais a decisão final não depende apenas da saída de um modelo estatístico, mas também de regras explícitas que refletem o conhecimento operacional. Essa abordagem permite que o sistema execute decisões automatizadas de forma estruturada e explicável, melhorando a confiabilidade e o controle sobre o processo de automação.

3.2 Contextualização do exemplo integrado

Um pipeline de automação inteligente em ambiente corporativo tende a operar sobre múltiplas fontes de entrada e múltiplos formatos documentais. Em processos administrativos, financeiros e de atendimento, é comum receber solicitações por arquivos de texto, formulários exportados, imagens digitalizadas e PDFs provenientes de scanners. Um robô de automação que opere apenas com regras determinísticas

tem dificuldade de tratar variabilidade linguística e não lida bem com entradas não estruturadas. O exemplo a seguir foi projetado para demonstrar como construir um pipeline offline que combine leitura de documentos, extração de evidências, classificação estatística e decisão inferencial baseada em regras, resultando em ações de RPA observáveis e auditáveis.

O componente de OCR entra no pipeline para transformar documentos não textuais em texto processável. Em ambientes reais, boa parte dos documentos críticos não possui camada textual confiável, exigindo reconhecimento óptico para que o sistema possa analisar o conteúdo. O Tesseract permite realizar OCR localmente, o que é importante em cenários com restrições de privacidade, compliance ou ausência de conectividade. A presença do OCR no pipeline também serve como ponto de discussão sobre qualidade de entrada, ruído e necessidade de validação das extrações.

O componente de Machine Learning é utilizado para tomar decisões probabilísticas a partir de padrões linguísticos aprendidos com exemplos. No exemplo, um classificador textual estima uma categoria operacional para o documento, como suporte, comercial ou financeiro, e fornece uma distribuição de probabilidades. Esse resultado permite que a automação não apenas escolha uma classe, mas também quantifique a confiança da decisão. Em uso futuro, essa confiança é um elemento relevante para escalonar casos incertos para revisão manual ou aplicar regras mais conservadoras em processos de risco.

O motor inferencial opera como camada de decisão explícita e explicável, transformando saídas estatísticas e evidências extraídas em uma ação operacional formal. Regras podem combinar categoria estimada, confiança e indícios estruturados, como valores monetários extraídos do texto, para inferir ações como auditoria, processamento, encaminhamento setorial ou revisão manual. Esse desenho representa uma arquitetura híbrida útil em organizações, pois permite adaptar políticas de decisão sem retreinar modelos. A rastreabilidade é reforçada pelo relatório gerado, que registra probabilidades, evidências e decisões, o que é importante para auditoria do próprio robô.

O uso do sistema de arquivos como mecanismo de entrada e saída é intencional e alinhado com práticas de RPA, pois muitas automações corporativas

operam com pastas de “inbox” e “outbox” integradas a processos internos, compartilhamentos e esteiras documentais. A movimentação de arquivos materializa a execução, enquanto os relatórios textuais formam uma trilha técnica para validação e melhoria contínua. Esse padrão é apropriado para ensino, pois torna o fluxo visível, reproduzível e simples de testar localmente.

3.3 Instalação e preparação do ambiente

A execução do exemplo exige bibliotecas Python e um componente de OCR instalado no sistema operacional. As bibliotecas Python podem ser instaladas via pip e permanecem executando localmente. O Tesseract é instalado como programa nativo, pois realiza o OCR fora do interpretador Python, sendo invocado pela biblioteca pytesseract.

Instale as dependências Python com o comando a seguir. O scikit-learn fornece vetorização e classificação, o joblib permite persistência do modelo, o experta fornece o motor inferencial, o pytesseract integra o OCR ao Python, o pillow permite manipular imagens e o pdf2image permite converter PDFs em imagens para OCR quando necessário.

```
pip install scikit-learn joblib experta pytesseract pdf2image pillow
```

Comando de prompt 3 - Instalação das bibliotecas (2)
Fonte: Elaborado pelo autor (2026)

Instale o Tesseract no sistema. Em distribuições Debian e Ubuntu é comum utilizar o gerenciador de pacotes. Em Windows, a instalação é feita via instalador e pode exigir configuração do caminho do executável, caso o sistema não o encontre automaticamente.

```
sudo apt install tesseract-ocr
```

Comando de prompt 4 - Instalação das bibliotecas (3)
Fonte: Elaborado pelo autor (2026)

Crie uma estrutura de diretórios para simular um fluxo de produção. A pasta de entrada recebe arquivos, as pastas de processado representam destinos operacionais e a pasta de relatórios armazena a trilha explicável do processo. O pipeline foi

desenhado para processar arquivos .txt, imagens e PDFs, desde que as dependências estejam disponíveis.

```
projeto_rpa_ia/  
  entrada/  
  processado/  
    suporte/  
    comercial/  
    financeiro/  
    auditoria/  
    manual/  
  relatorios/  
  modelos/  
  regras/  
    regras.py  
  pipeline.py
```

3.4 Código 1: motor inferencial com Experta

Crie o arquivo regras/regras.py com o conteúdo a seguir. Esse código define os fatos que descrevem o documento e as regras que inferem ações e prioridades com base em categoria, confiança e evidências extraídas.

```
from experta import KnowledgeEngine, Fact, Rule, P, MATCH  
  
class Documento(Fact):  
    """Fato principal com informações do documento  
    processado."""  
    pass  
  
class Decisao(Fact):  
    """Fatos inferidos pelo motor, usados para guiar  
    ações."""  
    pass  
  
class MotorInferencial(KnowledgeEngine):  
    """  
    Motor inferencial com forward chaining.  
    A base de regras usa categoria, confiança e evidências  
    extraídas (OCR/regex)
```

```
para inferir ações e níveis de prioridade.
"""

@Rule(Documento(categoria="financeiro",
                 confianca=P(lambda x: x >= 0.75),
                 valor_total=P(lambda v: v is not None
and v >= 10000)))
    def financeiro_alto_valor(self):
        self.declare(Decisao(acao="AUDITAR",
prioridade="ALTA",
                                motivo="Financeiro com valor
elevado e alta confiança"))

    @Rule(Documento(categoria="financeiro",
                 confianca=P(lambda x: x >= 0.60),
                 valor_total=P(lambda v: v is not None
and v >= 2000)))
    def financeiro_medio_valor(self):
        self.declare(Decisao(acao="PROCESSAR",
prioridade="MEDIA",
                                motivo="Financeiro com valor
moderado e confiança adequada"))

    @Rule(Documento(categoria="suporte",
                 confianca=P(lambda x: x >= 0.70)))
    def suporte_padrao(self):
        self.declare(Decisao(acao="ENCAMINHAR_SUPORTE",
prioridade="MEDIA",
                                motivo="Suporte identificado
com confiança adequada"))

    @Rule(Documento(categoria="comercial",
                 confianca=P(lambda x: x >= 0.70)))
    def comercial_padrao(self):
        self.declare(Decisao(acao="ENCAMINHAR_COMERCIAL",
prioridade="MEDIA",
                                motivo="Comercial
identificado com confiança adequada"))

    @Rule(Documento(confianca=P(lambda x: x < 0.60)))
    def baixa_confianca(self):
        self.declare(Decisao(acao="REVISAR_MANUAL",
prioridade="ALTA",
                                motivo="Baixa confiança do
modelo, revisão necessária"))

    @Rule(Documento(categoria="financeiro",
                 valor_total=P(lambda v: v is None)),
           salience=10)
    def financeiro_sem_valor(self):
```

```
        self.declare(Decisao(acao="REVISAR_MANUAL",
prioridade="ALTA",
                                motivo="Financeiro sem valor
extraído, possível falha de OCR"))

        @Rule(Documento(categoria=MATCH.cat,
                        confianca=MATCH.conf),
                Decisao(acao=MATCH.acao, prioridade=MATCH.prio,
motivo=MATCH.motivo))
        def consolidar(self, cat, conf, acao, prio, motivo):
            pass
```

Código-fonte 2 - Exemplo de código-fonte HTML
Fonte: Elaborado pelo autor (2019)

Esse arquivo define duas classes de fatos. A classe Documento representa o que o sistema sabe sobre o documento processado em um determinado ciclo do pipeline. A classe Decisao representa conclusões produzidas pelo motor a partir das regras, já no formato de ação operacional, incluindo prioridade e motivo para fins de explicabilidade. O motor MotorInferencial herda de KnowledgeEngine, o que habilita o ciclo de inferência interno do Expert. Cada regra é definida com o decorador Rule e descreve um padrão sobre fatos Documento. Quando o padrão é satisfeito, o método correspondente é executado e produz um fato Decisao por meio de declare.

As regras foram construídas para evidenciar a combinação de informações estatísticas e estruturadas. A primeira e a segunda regra demonstram um caso típico de compliance: documentos financeiros com evidência de valor elevado e confiança suficiente são encaminhados para auditoria ou processamento. As regras de suporte e comercial demonstram encaminhamento automático baseado em categoria e confiança. A regra de baixa confiança mostra um mecanismo de segurança operacional, em que o sistema se protege contra decisões incertas e encaminha o caso para revisão manual. A regra financeiro sem valor explora a relação entre OCR e inferência, permitindo que falhas de extração provoquem uma decisão conservadora. A regra consolidar foi incluída para estruturar o espaço de fatos e facilitar extensão futura, mantendo o sistema preparado para registrar trilhas de decisão quando se desejar ampliar o logging do motor.

3.5 Pipeline de automação com OCR, ML e inferência

Crie o arquivo pipeline.py com o seguinte conteúdo. Esse código implementa todo o fluxo de aquisição, extração, classificação, inferência, relatório e execução de ações no sistema de arquivos.

```
import os import re import shutil from dataclasses import
dataclass from typing import Optional, Tuple, List
import joblib from sklearn.feature_extraction.text import
TfidfVectorizer from sklearn.linear_model import
LogisticRegression
from regras.regras import MotorInferencial, Documento,
Decisao
import pytesseract from PIL import Image
try: from pdf2image import convert_from_path except
Exception: convert_from_path = None
PASTA_ENTRADA = "entrada" PASTA_PROCESSADO = "processado"
PASTA_RELATORIOS = "relatorios" PASTA_MODELOS = "modelos"
DESTINOS = { "suporte": os.path.join(PASTA_PROCESSADO,
"suporte"), "comercial": os.path.join(PASTA_PROCESSADO,
"comercial"), "financeiro": os.path.join(PASTA_PROCESSADO,
"financeiro"), "auditoria": os.path.join(PASTA_PROCESSADO,
"auditoria"), "manual": os.path.join(PASTA_PROCESSADO,
"manual"), }
MODELO_PATH = os.path.join(PASTA_MODELOS, "modelo.joblib")
VEC_PATH = os.path.join(PASTA_MODELOS, "vectorizer.joblib")
@dataclass class ResultadoML: categoria: str confianca:
float probs: List[Tuple[str, float]]
@dataclass class Extracao: valor_total: Optional[float]
data: Optional[str] cnpj: Optional[str]
def normalizar_texto(txt: str) -> str: txt = txt.lower()
txt = re.sub(r"\s+", " ", txt).strip() return txt
def extrair_campos(texto: str) -> Extracao: texto =
texto.replace("\n", " ")
m_valor = re.search(
r"r\$ *(\d{1,3} (?:\.\d{3}) * (?:,\d{2}) |\d+ (?:,\d{2}))",
texto,
flags=re.IGNORECASE
)
valor = None
if m_valor:
bruto = m_valor.group(1).replace(".", "").replace(",", "",
".")
try:
valor = float(bruto)
except ValueError:
valor = None
```

```
m_data = re.search(r"\b(\d{2}/\d{2}/\d{4})\b", texto)
data = m_data.group(1) if m_data else None

m_cnpj = re.search(r"\b(\d{2}\.\d{3}\.\d{3}/\d{4}-\d{2})\b",
texto)
cnpj = m_cnpj.group(1) if m_cnpj else None

return Extracao(valor_total=valor, data=data, cnpj=cnpj)

def ler_texto_de_arquivo(caminho: str) -> str: ext =
os.path.splitext(caminho)[1].lower()
    if ext == ".txt":
        with open(caminho, "r", encoding="utf-8") as f:
            return f.read()

if ext in [".png", ".jpg", ".jpeg"]:
    img = Image.open(caminho)
    return pytesseract.image_to_string(img, lang="por")

if ext == ".pdf":
    if convert_from_path is None:
        raise RuntimeError("pdf2image não disponível. Instale
pdf2image e poppler.")
    paginas = convert_from_path(caminho)
    partes = []
    for pg in paginas:
        partes.append(pytesseract.image_to_string(pg,
lang="por"))
    return "\n".join(partes)

raise ValueError(f"Tipo de arquivo não suportado: {ext}")

def treinar_se_nao_existir(): os.makedirs(PASTA_MODELOS,
exist_ok=True)
    if os.path.exists(MODELO_PATH) and
os.path.exists(VEC_PATH):
        return

dados = [
    "erro ao acessar o sistema de autenticação",
    "falha no login do usuário e token inválido",
    "sistema não responde e apresenta timeout",
    "problema ao instalar o aplicativo e erro de dependência",
    "instabilidade no servidor e falha de conexão",

    "pedido de orçamento para 50 licenças",
    "interesse em comprar plano empresarial",
    "solicitação de preços e condições comerciais",
    "cotação de serviço e proposta comercial",
    "informações sobre upgrade e contratação",
```

```
"segunda via de boleto e cobrança em aberto",
"confirmação de pagamento e comprovante anexado",
"erro na cobrança e divergência no valor",
"fatura mensal e vencimento em 10/02/2026",
"reembolso e estorno do pagamento"
]

labels = [
    "suporte", "suporte", "suporte", "suporte", "suporte",
    "comercial", "comercial", "comercial", "comercial",
    "comercial",
    "financeiro", "financeiro", "financeiro", "financeiro",
    "financeiro"
]

vec = TfidfVectorizer(ngram_range=(1, 2), min_df=1)
X = vec.fit_transform([normalizar_texto(t) for t in dados])

clf = LogisticRegression(max_iter=200)
clf.fit(X, labels)

joblib.dump(clf, MODELO_PATH)
joblib.dump(vec, VEC_PATH)

def classificar_texto(texto: str) -> ResultadoML:
    clf = joblib.load(MODELO_PATH)
    vec = joblib.load(VEC_PATH)
    X = vec.transform([normalizar_texto(texto)])
    probs = clf.predict_proba(X)[0]
    classes = clf.classes_

    pares = sorted([(c, float(p)) for c, p in zip(classes, probs)],
                    key=lambda x: x[1],
                    reverse=True)
    return ResultadoML(categoria=pares[0][0],
                        confianca=pares[0][1], probs=pares)

def inferir_decisao(categoria: str, confianca: float,
                    extracao: Extracao):
    motor = MotorInferencial()
    motor.reset()
    motor.declare(Documento(
        categoria=categoria,
        confianca=confianca,
        valor_total=extracao.valor_total,
        data=extracao.data,
        cnpj=extracao.cnpj
    ))

motor.run()

decisoes = []
for f in motor.facts.values():
```

```
    if isinstance(f, Decisao):
        decisoes.append(f)
return decisoes

def selecionar_destino(decisoes) -> str: if not decisoes:
return DESTINOS["manual"]
    prioridade_rank = {"ALTA": 3, "MEDIA": 2, "BAIXA": 1}
ordenadas = sorted(decisoes,
                    key=lambda d:
prioridade_rank.get(d["prioridade"], 0),
                    reverse=True)
acao = ordenadas[0]["acao"]

if acao == "AUDITAR":
    return DESTINOS["auditoria"]
if acao == "PROCESSAR":
    return DESTINOS["financeiro"]
if acao == "ENCAMINHAR_SUPOORTE":
    return DESTINOS["suporte"]
if acao == "ENCAMINHAR_COMERCIAL":
    return DESTINOS["comercial"]
return DESTINOS["manual"]

def gerar_relatorio(nome_arquivo: str, texto: str, ml:
ResultadoML, extr: Extracao, decisoes, destino: str):
os.makedirs(PASTA_RELATORIOS, exist_ok=True) rel_path =
os.path.join(PASTA_RELATORIOS,
f"{nome_arquivo}.relatorio.txt")
    with open(rel_path, "w", encoding="utf-8") as f:
        f.write("RELATÓRIO DE PROCESSAMENTO AUTOMATIZADO\n")
        f.write(f"Arquivo: {nome_arquivo}\n")
        f.write(f"Destino operacional: {destino}\n\n")

        f.write("SAÍDA DO MODELO (ML)\n")
        f.write(f"Categoria escolhida: {ml.categoria}\n")
        f.write(f"Confiança estimada: {ml.confianca:.3f}\n")
        f.write("Distribuição de probabilidades:\n")
        for c, p in ml.probs:
            f.write(f"    {c}: {p:.3f}\n")
        f.write("\n")

        f.write("EVIDÊNCIAS EXTRAÍDAS (OCR/Regex)\n")
        f.write(f"Valor total: {extr.valor_total}\n")
        f.write(f>Data: {extr.data}\n")
        f.write(f"CNPJ: {extr.cnpj}\n\n")

        f.write("DECISÕES INFERIDAS (MOTOR DE REGRAS)\n")
        if not decisoes:
            f.write("Nenhuma decisão inferida. Encaminhado para
revisão manual.\n")
        else:
```

```
        for d in decisoes:
            f.write(f"- acao={d['acao']}\n")
            f.write(f"prioridade={d['prioridade']} motivo={d['motivo']}\n")
            f.write("\n")

        f.write("TRECHO DO TEXTO PROCESSADO (AMOSTRA)\n")
        f.write(normalizar_texto(texto)[:600] + "\n")

    def processar_arquivo(caminho: str): nome =
os.path.basename(caminho) base = os.path.splitext(nome)[0]
        texto = ler_texto_de_arquivo(caminho)
        extr = extrair_campos(texto)

    ml = classificar_texto(texto)
    decisoes = inferir_decisao(ml.categoria, ml.confianca, extr)

    destino = selecionar_destino(decisoes)
    os.makedirs(destino, exist_ok=True)

    gerar_relatorio(base, texto, ml, extr, decisoes, destino)
    shutil.move(caminho, os.path.join(destino, nome))

    print(f"\nPROCESSADO: {nome}")
    print(f"Categoria: {ml.categoria} Confiança:
{ml.confianca:.3f}")
    print(f"Valor: {extr.valor_total} Data: {extr.data} CNPJ:
{extr.cnpj}")
    print(f"Destino: {destino}")

    def preparar_pastas(): os.makedirs(PASTA_ENTRADA,
exist_ok=True) os.makedirs(PASTA_PROCESSADO, exist_ok=True)
    for p in DESTINOS.values(): os.makedirs(p, exist_ok=True)
    os.makedirs(PASTA_RELATORIOS, exist_ok=True)
    os.makedirs(PASTA_MODELOS, exist_ok=True)

    def main(): preparar_pastas() treinar_se_nao_existir()
        arquivos = [os.path.join(PASTA_ENTRADA, a) for a in
os.listdir(PASTA_ENTRADA)]
        arquivos = [a for a in arquivos if os.path.isfile(a)]

    if not arquivos:
        print("Nenhum arquivo em 'entrada/'. Adicione .txt, .png,
.jpg, .jpeg ou .pdf e execute novamente.")
        return

    for caminho in arquivos:
        try:
            processar_arquivo(caminho)
        except Exception as e:
            print(f"\nERRO ao processar
{os.path.basename(caminho)}: {e}")
            os.makedirs(DESTINOS["manual"], exist_ok=True)
```

```
shutil.move(caminho, os.path.join(DESTINOS["manual"],  
os.path.basename(caminho)))  
  
if name == "main": main()
```

Código-fonte 3 - Pipeline Completo
Fonte: Elaborado pelo autor (2019)

Esse pipeline começa estabelecendo configurações de diretórios e caminhos de artefatos de modelo, pois em automação corporativa é importante que a entrada, saída e rastreabilidade sejam padronizadas. A definição de dataclasses organiza a saída do modelo e o resultado de extrações, evitando estruturas soltas e facilitando leitura e manutenção, o que é uma prática comum em sistemas de automação que precisam evoluir com segurança. As funções `normalizar_texto` e `extrair_campos` representam a etapa de pré-processamento e extração de evidências, o que permite ao sistema tratar o mesmo documento sob dois pontos de vista, um estatístico e outro estruturado, reduzindo dependência de um único mecanismo de decisão.

A função `ler_texto_de_arquivo` demonstra a padronização de entrada. Arquivos de texto são lidos diretamente, enquanto imagens são convertidas em texto via OCR e PDFs passam por conversão para imagem e OCR em cada página. Esse desenho é comum em pipelines documentais e permite que o restante do processo ignore o formato original, recebendo sempre texto. A função `treinar_se_nao_existir` implementa persistência de modelo local, treinando apenas quando os arquivos de modelo não existem. Essa estratégia evita retreinamento a cada execução do robô e garante reprodutibilidade, pois o mesmo modelo será aplicado aos mesmos dados até que se decida substituí-lo.

A função `classificar_texto` materializa a decisão estatística. Ela carrega o modelo persistido, vetorizador e calcula probabilidades por classe, retornando tanto a categoria escolhida quanto a confiança e a distribuição completa. Essa distribuição é registrada em relatório para tornar o processo observável e para permitir calibração futura do limiar de confiança. A função `inferir_decisao` representa a transição para a decisão simbólica. Ela converte a saída do modelo e as evidências extraídas em fatos do motor, executa o motor e coleta as decisões inferidas, que carregam ação, prioridade e motivo. A função `selecionar_destino` transforma a decisão inferida em consequência operacional concreta, escolhendo um destino de pasta de acordo com a ação mais crítica.

A função `gerar_relatorio` implementa explicabilidade prática no nível do robô. Em vez de registrar apenas a saída final, ela registra distribuição de probabilidade, evidências extraídas e decisões inferidas com motivos. Esse relatório permite auditoria posterior e facilita o ensino, pois o aluno consegue verificar claramente como a decisão foi formada e como a automação executou a ação. A função `processar_arquivo` integra todas as etapas: leitura, extração, classificação, inferência, geração de relatório e movimentação do arquivo. O método `main` fecha o pipeline, preparando pastas, garantindo que o modelo exista, listando arquivos de entrada e aplicando o processamento a cada item.

CONCLUSÃO

A integração entre técnicas de Machine Learning, reconhecimento óptico de caracteres e motores inferenciais representa um avanço significativo na construção de sistemas de hiperautomação capazes de operar sobre dados não estruturados e tomar decisões automatizadas com base em múltiplas fontes de evidência. O uso combinado dessas tecnologias permite que sistemas de RPA deixem de executar apenas tarefas determinísticas e passem a incorporar mecanismos de análise e raciocínio, ampliando sua capacidade de atuação em cenários reais, nos quais a variabilidade e a incerteza são elementos predominantes.

O modelo de classificação estatística apresentado neste capítulo demonstra como padrões linguísticos podem ser utilizados para estimar categorias operacionais de documentos de forma automatizada. A utilização de vetorização textual e algoritmos de aprendizado supervisionado permite transformar conteúdo textual em representações numéricas que podem ser analisadas computacionalmente. Esse processo constitui a base da tomada de decisão estatística, permitindo que o sistema estime não apenas uma categoria, mas também um nível de confiança associado, o que é fundamental para a construção de sistemas automatizados confiáveis.

A introdução do motor inferencial acrescenta uma camada de raciocínio simbólico, que permite interpretar os resultados estatísticos dentro de um contexto operacional definido por regras explícitas. Essa abordagem permite separar a etapa de estimativa probabilística da etapa de decisão operacional, proporcionando maior controle sobre o comportamento do sistema. A utilização de regras declarativas permite adaptar o sistema a novos requisitos operacionais sem a necessidade de modificar ou retreinar o modelo estatístico, facilitando a manutenção e evolução da automação.

O pipeline implementado demonstra como essas tecnologias podem ser integradas em um fluxo completo de automação, incluindo aquisição de documentos, extração de informações, classificação automática, inferência baseada em regras e execução de ações no sistema de arquivos. A geração de relatórios detalhados permite registrar cada etapa do processo, contribuindo para a explicabilidade e auditabilidade do sistema. Essa capacidade de rastrear decisões é especialmente

relevante em ambientes corporativos, onde a transparência e a confiabilidade dos sistemas automatizados são requisitos fundamentais.

A arquitetura apresentada neste capítulo estabelece uma base sólida para o desenvolvimento de sistemas de automação inteligente capazes de operar de forma autônoma, explicável e adaptável. A combinação de aprendizado de máquina e inferência baseada em regras permite construir sistemas que integram análise estatística e raciocínio lógico, ampliando significativamente as capacidades da automação tradicional. Esse modelo representa um passo importante na evolução da hiperautomação e fornece os fundamentos necessários para o desenvolvimento de soluções mais avançadas, capazes de suportar processos complexos em diferentes domínios organizacionais.

REFERÊNCIAS

CAROLA, Gabriel Alves. **Machine learning aplicado a sistemas inteligentes**. Goiânia: PUC Goiás, 2024.

DOMINGOS, Pedro. **The master algorithm**: how the quest for the ultimate learning machine will remake our world. New York: Basic Books, 2015.

GÉRON, Aurélien. **Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow**: concepts, tools, and techniques to build intelligent systems. 2. ed. Sebastopol: O'Reilly Media, 2019.

GIARRATANO, Joseph; RILEY, Gary. **Expert systems**: principles and programming. 4. ed. Boston: Cengage Learning, 2004.

LUGER, George. **Artificial intelligence**: structures and strategies for complex problem solving. 6. ed. Boston: Pearson, 2009.

MORAES, Carlos Henrique Vieira *et al.* Robotic process automation and machine learning: a systematic review. **Brazilian Archives of Biology and Technology**, Curitiba, v. 65, 2022.

NASCIMENTO, João Victor. **Automação de processos com robotic process automation**. Editora Científica Digital, 2025.

NORVIG, Peter; RUSSELL, Stuart. **Artificial intelligence**: a modern approach. 4. ed. Upper Saddle River: Pearson, 2020.

REIS, L. J. P. F. *et al.* Uma revisão sistemática da literatura sobre robotic process automation (RPA). **Estudo & Debate**, Lajeado, v. 31, n. 1, 2024.

RIBEIRO, Tânia de Souza. **Automatização de processos com uso de robotic process automation**. São Paulo: Centro Paula Souza, 2023.

RUDIN, Cynthia *et al.* Interpretable machine learning: fundamental principles and grand challenges. **arXiv preprint**, [S. l.], 2021. Disponível em: <https://arxiv.org>. Acesso em: 24 fev. 2026.

SMITH, Ray. An overview of the Tesseract OCR engine. *In*: INTERNATIONAL CONFERENCE ON DOCUMENT ANALYSIS AND RECOGNITION, 9, 2007. **Proceedings [...]**. [S. l.]: IEEE, 2007.

SOKOL, Kacper; FLACH, Peter. Explainability fact sheets: a framework for systematic assessment of explainable approaches. **arXiv preprint**, [S. l.], 2019. Disponível em: <https://arxiv.org>. Acesso em: 24 fev. 2026.

VILONE, Giulia; LONGO, Luca. Explainable artificial intelligence: a systematic review. **arXiv preprint**, [S. l.], 2020. Disponível em: <https://arxiv.org>. Acesso em: 24 fev. 2026.

WINSTON, Patrick Henry. **Artificial intelligence**. 3. ed. Reading: Addison-Wesley, 1992.

EVANS